

Dynamické struktury v C

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© richta, 2023

Základy strukturovaného programování,
ZSP

11/2023, Lekce 11

<https://moodle.vspj.cz/course/view.php?id=202262>



Vysoká škola
polytechnická
Jihlava



Co jsou dynamické struktury?

- Dynamické struktury jsou datové struktury, které umožňují flexibilní manipulaci s pamětí během běhu programu. Na rozdíl od statických struktur, které mají pevně stanovenou velikost při kompilaci, dynamické struktury mohou být vytvářeny a měněny za běhu programu.
- Příklady dynamických struktur v jazyce C zahrnují pole, spojové seznamy, fronty, zásobníky, stromy, atd.

Práce s dynamickými strukturami v jazyce C

Alokace paměti

- Alokace paměti pro dynamické struktury se provádí pomocí funkcí malloc, calloc nebo realloc. Funkce malloc alokuje určitý počet bajtů paměti, zatímco calloc alokuje paměť pro určitý počet prvků dané velikosti. Funkce realloc slouží k změně velikosti již alokované paměti.

```
int *pole = (int*)malloc(5*sizeof(int)) ;
```

Uvolňování paměti

- Je důležité uvolňovat paměť, kterou dynamicky alokujeme, aby nedocházelo k paměťovým únikům. K tomu slouží funkce free.

```
free(pole) ;
```

Práce s dynamicky alokovaným polem

- Práce s polem v jazyce C zahrnuje používání ukazatelů a aritmetiky ukazatelů. Dynamicky alokované pole může být používáno obdobně jako staticky alokované pole.

```
int *pole = (int*)malloc(5*sizeof(int)) ;  
for (int i = 0; i < 5; ++i) {  
    pole[i] = i * 2;  
}
```

Problém se spojovými strukturami

```
/* definice uživatelského typu uzel ve spojovém seznamu */  
int f(int x) {  
    int *pole = (int*)malloc(5*sizeof(int));  
    return x;  
}
```

```
f(10);
```

Chybí free

Práce se spojovými strukturami

- Spojové struktury jsou paměťové struktury provázané pomocí ukazatelů. Triviální případ jsou lineární spojové seznamy.
- Spojový seznam je dynamická datová struktura, která umožňuje přidávání a odebírání prvků za běhu programu. Každý prvek obsahuje data a odkaz na následující prvek.

Grafy

- **Graf** je struktura, která nám pomáhá znázorňovat objekty a vztahy mezi nimi. Skládá se z **vrcholů** a **hran**. Vrcholy často reprezentují reálné objekty a obvykle je kreslíme jako tečky nebo kolečka. Hrany představují vztahy mezi vrcholy, na obrázku obvykle vypadají jako čáry mezi vrcholy. Každá hrana vede mezi dvěma vrcholy, oba její konce tedy musí být připojeny k některému vrcholu.

Základní grafové pojmy

Mezi základní grafové pojmy patří:

- **Stupeň** vrcholu je počet hran, které z daného vrcholu vychází.
- **Cesta** mezi dvěma určenými vrcholy existuje, pokud v grafu dokážeme přejít po hranách z jednoho vrcholu do druhého.
- **Vzdálenost** dvou vrcholů je délka nejkratší cesty mezi těmito vrcholy. Naopak nám vůbec nezáleží na tom, jak daleko jsou od sebe vrcholy na obrázku, pokud graf nakreslíme.

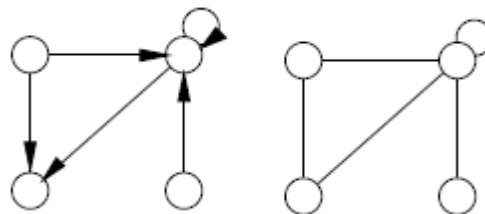
Základní grafové pojmy

- V **orientovaném grafu** mají hrany přesně určený směr, kterým vedou, a tedy i začáteční a koncový vrchol. To je rozdíl od grafů, kde hrany vedou „mezi vrcholy“ a nemají dáno, kde začínají a kde končí. Hrany orientovaných grafů se často znázorňují jako šipky.
- V **ohodnoceném grafu** má každá hrana přiřazenu určitou hodnotu (nazývanou také váha). V obrázku píšeme váhy jako čísla ke hranám. Pomocí těchto hodnot můžeme snadno znázornit například délky cest mezi městy.

Orientované a neorientované grafy

Definice

- **Orientovaný graf** o n uzlech je dvojice $G = (V(G), E(G))$, kde
 - ▶ $V(G)$ je **množina uzlů**, $n = |V(G)|$, a
 - ▶ $E(G) = \{\langle u, v \rangle; u, v \in V(G)\} \subset V(G) \times V(G) =$ **množina orientovaných hran**, čili **uspořádaných dvojic uzlů** (graficky šipek).
 - ▶ Orientovaná hrana $\langle u, u \rangle$ se nazývá **smyčka**.
- **(Neorientovaný obyčejný) graf** o n uzlech je dvojice $G = (V(G), E(G))$, kde $V(G)$ je **množina uzlů** a
 - ▶ $E(G) = \{\{u, v\}; u, v \in V(G), u \neq v\}$
= **množina neorientovaných hran**, čili **neuspořádaných dvojic uzlů**.
 - ▶ Smyčky se někdy povolují, někdy ne..



Reprezentace grafu v počítači

seznamy sousedů

- pro každý vrchol seznam jeho sousedů
- vhodné pro řídké matice

matice souslednosti

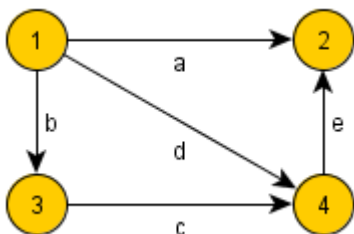
- binární matice
- pro každou dvojici vrcholů:
sousedí (1) / nesousedí (0)
- vhodné pro malé nebo husté matice

Příklad reprezentace orientovaného grafu

- **Matice sousednosti** je matice $U \times U$, ve které je souřadnice daná řádkem m a sloupcem n jednotková právě tehdy když z uzlu m vede hrana do uzlu n , v opačném případě je nulová. Matice sousednosti je pro neorientované grafy symetrická podle diagonály.
- **Matice incidence** - řádky matice incidence reprezentují uzly grafu, sloupce jednotlivé hrany. Souřadnice daná řádkem m a sloupcem n je 1, pokud je uzel m zdrojovým uzlem hrany n , -1 v případě, že je uzel cílovým uzlem hrany n a 0 v případě, že uzel m s hranou neinciduje.

Příklad reprezentace orientovaného grafu

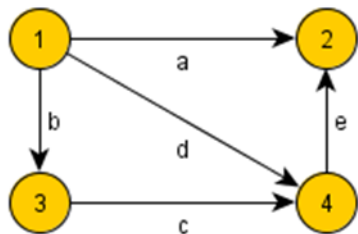
- **Matice sousednosti** je matice $U \times U$, ve které je souřadnice daná řádkem m a sloupcem n jednotková právě tehdy když z uzlu m vede hrana do uzlu n , v opačném případě je nulová. Matice sousednosti je pro neorientované grafy symetrická podle diagonály.



$$\begin{pmatrix} & \textcolor{red}{1} & \textcolor{red}{2} & \textcolor{red}{3} & \textcolor{red}{4} \\ \textcolor{red}{1} & 0 & 1 & 1 & 1 \\ \textcolor{red}{2} & 0 & 0 & 0 & 0 \\ \textcolor{red}{3} & 0 & 0 & 0 & 1 \\ \textcolor{red}{4} & 0 & 1 & 0 & 0 \end{pmatrix}$$

Příklad reprezentace orientovaného grafu

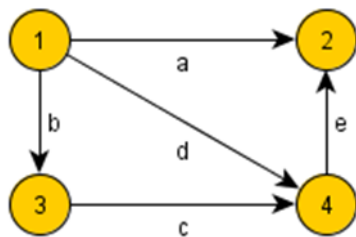
- **Matice incidence** - řádky matice incidence reprezentují uzly grafu, sloupce jednotlivé hrany. Souřadnice daná řádkem m a sloupcem n je 1, pokud je uzel m zdrojovým uzlem hrany n , -1 v případě, že je uzel cílovým uzlem hrany n a 0 v případě, že uzel m s hranou neinciduje.



$$\begin{pmatrix} & a & b & c & d & e \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 2 & -1 & 0 & 0 & 0 & -1 \\ 3 & 0 & -1 & 1 & 0 & 0 \\ 4 & 0 & 0 & -1 & -1 & 1 \end{pmatrix}$$

Spojová reprezentace grafu

Spojová reprezentace je patrně nejpoužívanějším způsobem reprezentace grafu. Má-li graf m uzlů, pak spojová reprezentace obsahuje m seznamů, každý z těchto seznamů obsahuje ukazatele na všechny uzly, do kterých vede hrana z uzlu m .



1 → 2,3,4

2 →

3 → 4

4 → 2

Práce se spojovým seznamem

```
/* definice uživatelského typu uzel ve spojovém seznamu */
typedef struct Vrchol {
    int cilovyVrchol;
    struct Vrchol* dalsi;
} Vrchol;

/* struktura pro reprezentaci grafu */
struct Graf {
    int pocetVrcholu;
    struct Vrchol** seznamVrcholu; // ukazatel na seznam uzlů
};

/* funkce pro vytvoření nového vrcholu s daným cílovým vrcholem */
struct Vrchol* vytvorVrchol(int cilovyVrchol) {
    struct Vrchol* novyVrchol =
        (struct Vrchol*)malloc(sizeof(struct Vrchol));
    novyVrchol->cilovyVrchol = cilovyVrchol;
    novyVrchol->dalsi = NULL;
    return novyVrchol;
}
```


Graf reprezentovaný spojovým seznamem

```
// Funkce pro vytvoření grafu s daným počtem vrcholů
struct Graf* vytvorGraf(int pocetVrcholu) {
    struct Graf* graf =
        (struct Graf*)malloc(sizeof(struct Graf));
    graf->pocetVrcholu = pocetVrcholu;

    // Vytvoř spojový seznam pro každý vrchol
    graf->seznamVrcholu =
        (struct Vrchol**)malloc(pocetVrcholu * sizeof(struct Vrchol*));
    for (int i = 0; i < pocetVrcholu; i++) {
        graf->seznamVrcholu[i] = NULL;
    }

    return graf;
}
```

Přidání hrany do grafu

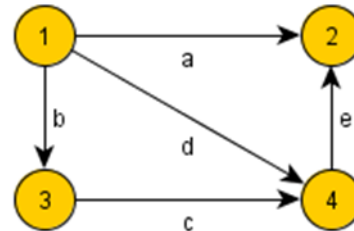
```
// Funkce pro přidání hrany do grafu
void pridatHranu(struct Graf* graf,
                 int zdrojovyVrchol,
                 int cilovyVrchol) {
    // Přidej hranu z vrcholu zdrojovyVrchol do cilovyVrchol
    struct Vrchol* novyVrchol = vytvorVrchol(cilovyVrchol);
    novyVrchol->dalsi = graf->seznamVrcholu[zdrojovyVrchol];
    graf->seznamVrcholu[zdrojovyVrchol] = novyVrchol;
}
```

Výpis grafu

```
// Funkce pro výpis grafu
void vypisGraf(struct Graf* graf) {
    for (int i = 0; i < graf->pocetVrcholu; i++) {
        struct Vrchol* aktualniVrchol = graf->seznamVrcholu[i];
        printf("Vrchol %d: ", i);
        while (aktualniVrchol != NULL) {
            printf("%d ", aktualniVrchol->cilovyVrchol);
            aktualniVrchol = aktualniVrchol->dalsi;
        }
        printf("\n");
    }
}
```

Výpis grafu

```
int main() {  
    int pocetVrcholu = 4;  
    struct Graf* graf = vytvorGraf(pocetVrcholu);  
  
    pridatHranu(graf, 1, 2);  
    pridatHranu(graf, 1, 3);  
    pridatHranu(graf, 1, 4);  
    pridatHranu(graf, 3, 4);  
    pridatHranu(graf, 4, 2);  
  
    vypisGraf(graf);  
  
    return 0;  
}
```



```
Vrchol 1: 4 3 2  
Vrchol 2:  
Vrchol 3: 4  
Vrchol 4: 2
```

Vlastnosti grafů

- Graf je **souvislý**, pokud mezi každými dvěma z jeho vrcholů vede cesta. To znamená, že všechny vrcholy jsou spolu nějak propojené – dokážeme v grafu přejít po hranách z každého vrcholu do všech ostatních.
- **Komponenta souvislosti** je část grafu, která je souvislá, ale pokud bychom do ní chtěli zahrnout nějaké další hrany nebo vrcholy, souvislá by být přestala. Každý graf je rozdělený na několik komponent souvislosti. Pokud je graf souvislý, tvoří sám o sobě jednu komponentu souvislosti.

Vlastnosti grafů

- **Podgraf** je část (tedy některé vybrané vrcholy a hrany) grafu, která sama o sobě také tvoří graf. Každá hrana v podgrafu tedy musí mít na obou svých koncích vrchol, který také patří do podgrafu.
- V **úplném grafu** je každý vrchol je spojený s každým. Tento graf má tedy maximální počet hran, který může mít.
- **Strom** je souvislý graf, který neobsahuje žádnou kružnici jako podgraf. Stromy mají mnoho zajímavých vlastností a často se používají v informatice, například pro přehledné a efektivní uložení dat.

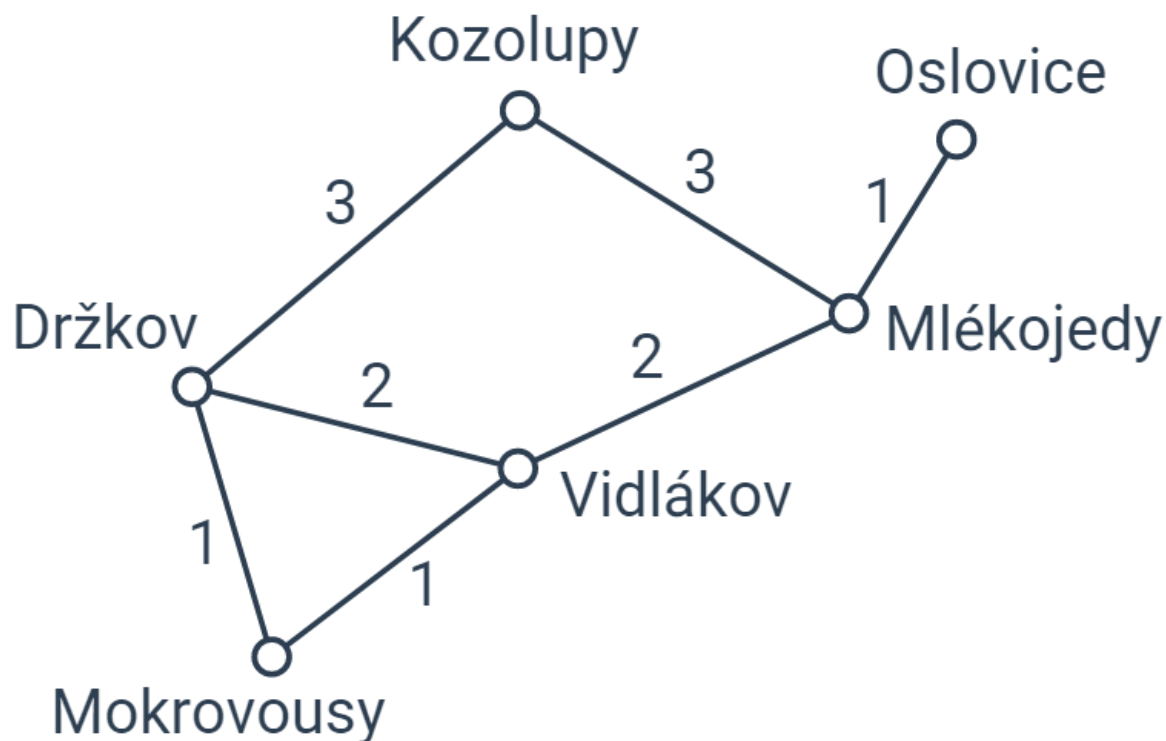
Hledání cesty v grafu

- Jednou z typických úloh na grafech je hledání cest mezi vrcholy. Pokud chceme v mapě najít co nejkratší cestu z jednoho místa do jiného, můžeme mapu převést na graf, v němž budeme hledat nejkratší cestu po hranách. V takové situaci se často hodí k hranám (cestám mezi místy) doplnit údaje o jejich délce.
- Cesty je možné kromě prostorové vzdálenosti porovnávat i podle jiných kritérií.
- Navigace neporovnává cesty jen podle délky, ale i podle času dojezdu. Cesta po dálnici pravděpodobně bude o mnoho rychlejší, než stejně dlouhá cesta po okresní silnici.

Příklad:

Kolik kilometrů má cesta z Kozolupy do Mlékojed?

Obrázek je zjednodušená mapa. Dvě místa jsou spojená, pokud mezi nimi vede cesta.
Čísla u cest jsou vzdálenosti v kilometrech.



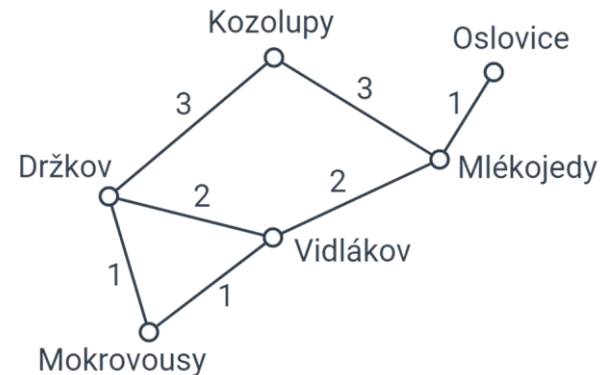
Přidání ohodnocené hrany do grafu

```
/* definice uživatelského typu uzel s cenou ve spojovém seznamu */
typedef struct Vrchol {
    int cilovyVrchol;
    double cena;
    struct Vrchol* dalsi;
} Vrchol;

/* Funkce pro přidání ohodnocené hrany do grafu */
void pridatHranu(struct Graf* graf,
                 int zdrojovyVrchol,
                 int cilovyVrchol,
                 double cena) {
    // Přidej hranu z vrcholu zdrojovyVrchol do cilovyVrchol
    struct Vrchol* novyVrchol = vytvorVrchol(cilovyVrchol);
    novyVrchol->dalsi = graf->seznamVrcholu[zdrojovyVrchol];
    novyVrchol->dalsi = graf->seznamVrcholu[zdrojovyVrchol];
    graf->seznamVrcholu[zdrojovyVrchol] = novyVrchol;
}
```

Výpis grafu

```
int main() {  
    int pocetVrcholu = 6;  
  
    const char *jmena[] = {  
        "Držkov", "Kozolupy", "Mlékovice", "Oslovice", "Mokrovousy", "Vidlákov";  
    };  
  
    struct Graf* graf = vytvorGraf(pocetVrcholu);  
  
    pridatHranu(graf, 1, 2, 3);  
    pridatHranu(graf, 1, 5, 1);  
    pridatHranu(graf, 1, 6, 2);  
    pridatHranu(graf, 2, 3, 3);  
    pridatHranu(graf, 3, 4, 1);  
    pridatHranu(graf, 3, 6, 2);  
    pridatHranu(graf, 6, 4, 1);  
  
    vypisGraf(graf);  
  
    return 0;  
}
```



Vrchol 1: 6 5 2

Vrchol 2: 3

Vrchol 3: 6 4

Vrchol 4:

Vrchol 5:

Vrchol 6: 4

Problém hledání cesty

- Řekněme, že máte mapu. Víte, kde jste a kam chcete jít. Mapa má silnice (reprezentujeme jako hrany), které spojují uzly (místa se souřadnicemi).
- Z každého uzlu můžete přejít na jednu nebo více hran. Hrana má náklady (např. délku nebo dobu potřebnou k jejímu překonání).
- U malých map by snad bylo možné spočítat všechny možné trasy do cíle a vybrat tu nejkratší. Ale to není příliš praktické pro mapy s mnoha uzly, protože kombinace rostou exponenciálně.

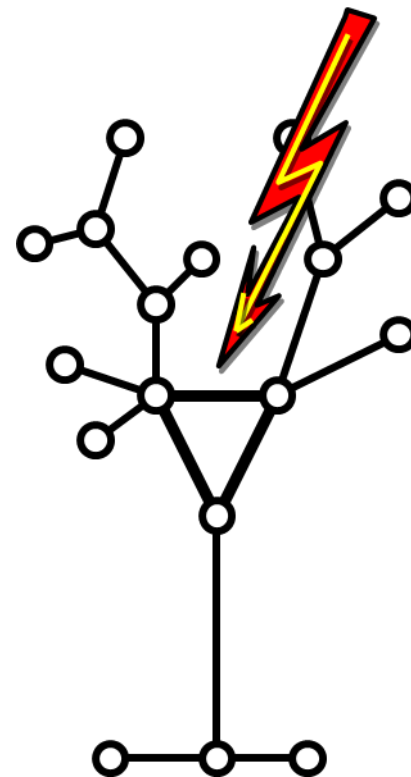
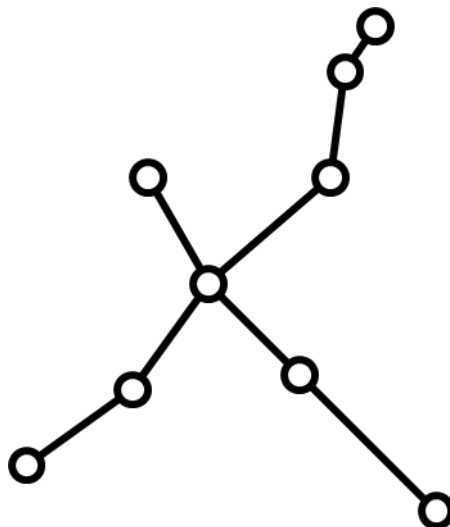
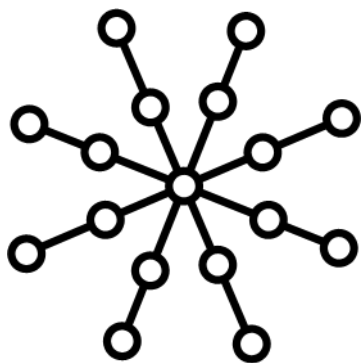
Dijkstrův algoritmus (1959)

1. Z počátečního uzlu přidejte všechny připojené uzly do prioritní fronty.
2. Seřadte prioritní frontu podle nejnižších nákladů a nastavte první uzel jako aktuální uzel.
3. Pro každý podřízený uzel vyberte nejlepší (co vede k nejkratší cestě).
4. Když byly všechny hrany prozkoumány z uzlu, je tento uzel "Navštíven" a nemusíte tam znovu chodit.
5. Přidejte každý podřízený uzel připojený k aktuálnímu uzlu do fronty priorit.
6. Přejděte ke kroku 2, dokud nebude fronta prázdná.
7. Rekurzivně vytvořte seznam všech uzlů, které vedou nejkratší cestu od konce k začátku.
8. Otočte seznam a našli jste nejkratší cestu
9. Jinými slovy, rekurzivně pro každého potomka uzlu změřte jeho vzdálenost k začátku. Uložte vzdálenost a to, který uzel vedl k nejkratší cestě ke startu. Když dosáhnete koncového uzlu, rekurzivně se vraťte na začátek nejkratší cestou, otočte tento seznam a máte nejkratší cestu.

Definice stromu (volného stromu)

Definice

- **Volný strom** je každý souvislý acyklický a neorientovaný graf.
- **Les** je každý acyklický a neorientovaný graf.

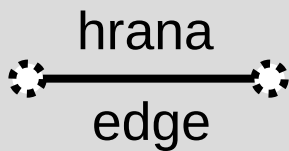


Strom

uzel, vrchol
node, vertex

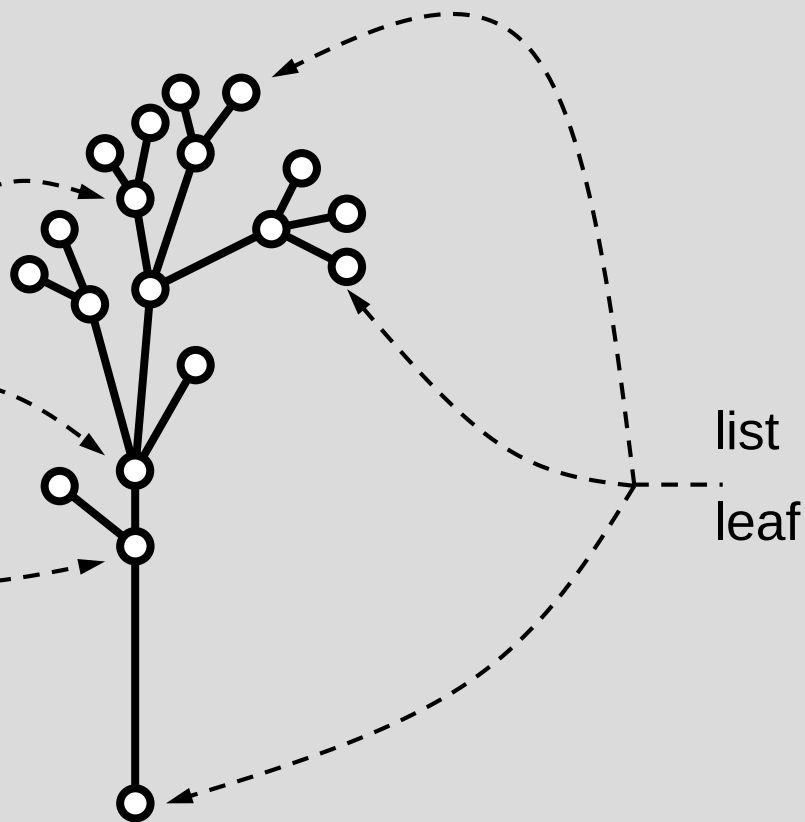


strom tree



vnitřní uzel
internal node

list
leaf



Vlastnosti volných stromů

Věta

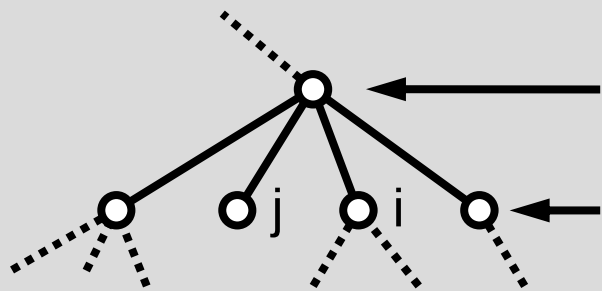
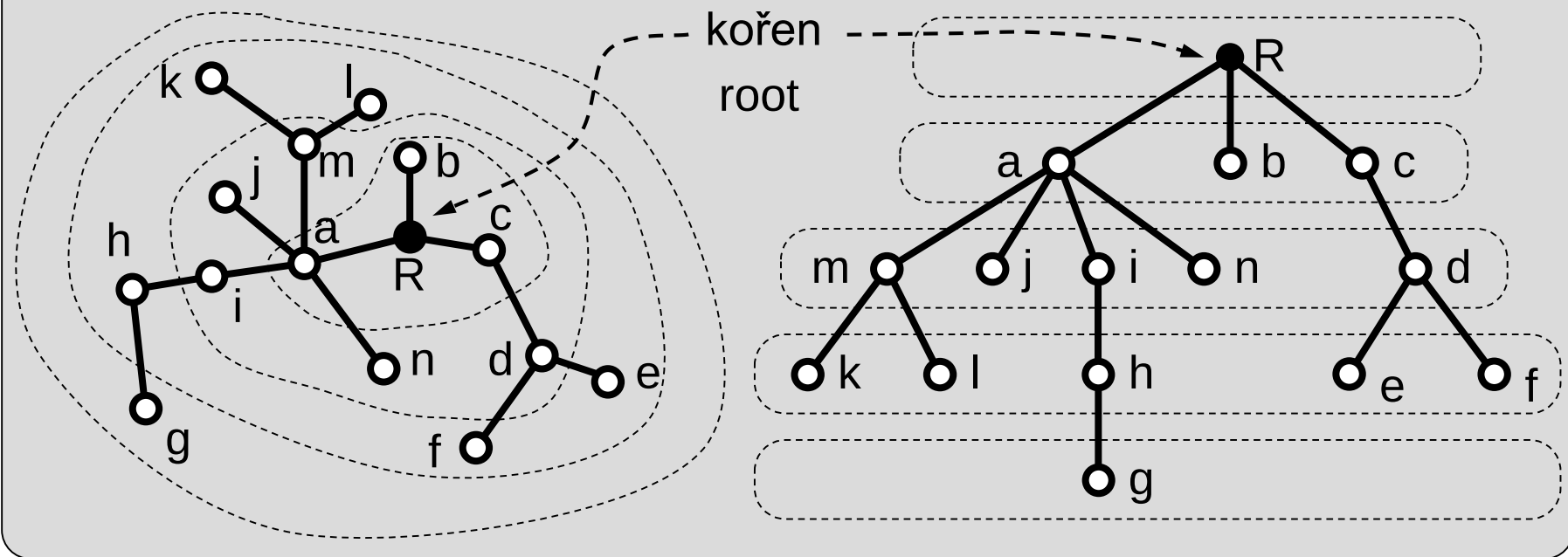
Nechť $G = (V(G), E(G))$ je neorientovaný graf. Pak následující tvrzení jsou ekvivalentní.

- ① *G je volný strom.*
- ② *Jakékoli 2 uzly v G jsou spojeny jedinečnou jednoduchou cestou.*
- ③ *G je souvislý, ale pokud vyjmeme jakoukoli hranu z $E(G)$, výsledný graf bude nesouvislý.*
- ④ *G je souvislý a $|E(G)| = |V(G)| - 1$.*
- ⑤ *G je acyklický a $|E(G)| = |V(G)| - 1$.*
- ⑥ *G je acyklický, ale pokud přidáme jakoukoli hranu do $E(G)$, výsledný graf bude obsahovat aspoň jeden cyklus.*

Kořenové stromy

kořenový strom

rooted tree



předchůdce, rodič

predecessor , parent

následník, potomek

successor, child

Definice kořenového stromu

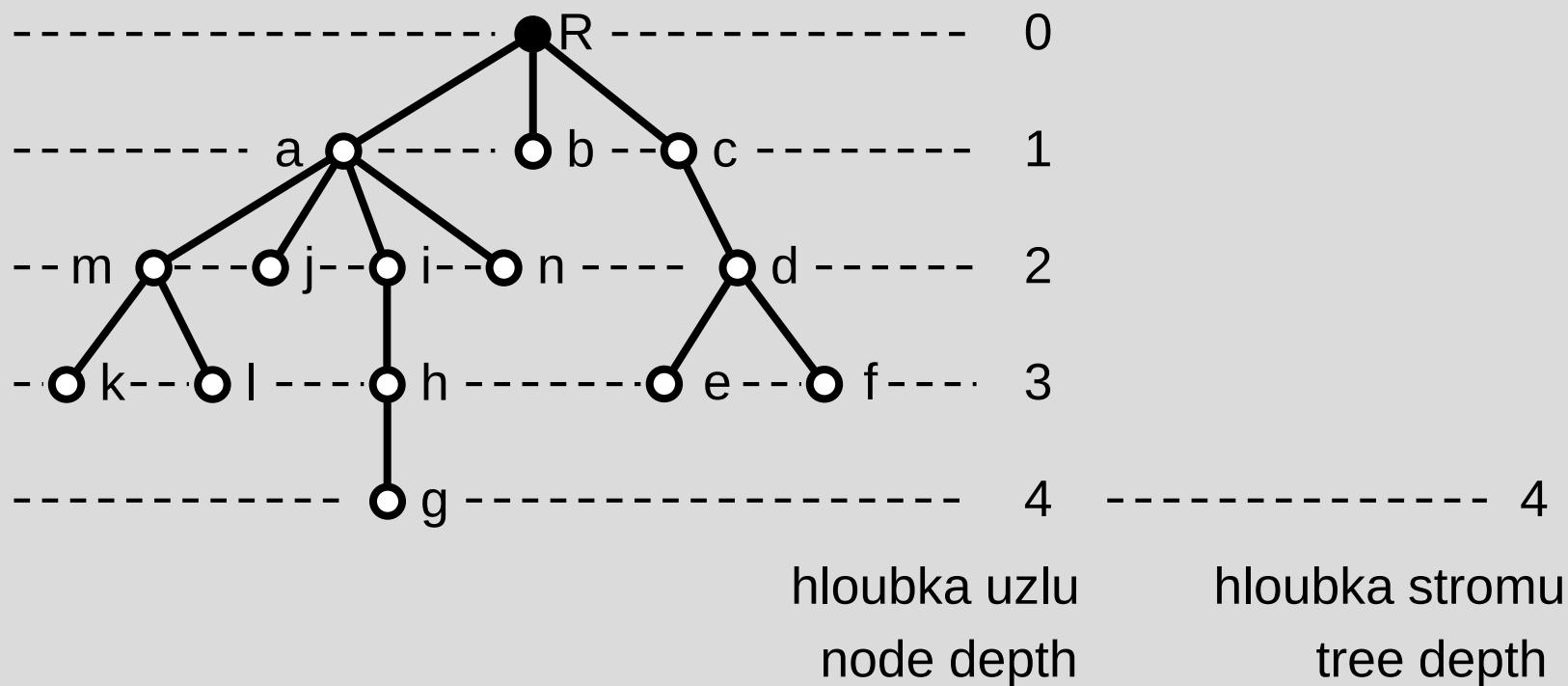
Definice

- **Kořenový strom** je volný strom, ve kterém jeden z uzlů je odlišen od ostatních jako **kořen**, r .
- Uzly u ve vzdálenosti d od kořenu r tvoří hladinu uzlů v **hloubce** $h(u) = d$. Kořen má hloubku $h(r) = 0$.
- Necht' uzel u leží na (jedinečné) cestě z kořene r do uzlu v . Pak u je **předek** v a v je **potomek** u .
- Nejblížeší předek uzlu je jeho **rodič** a nejblížeší potomek je jeho **syn**.
 $\Rightarrow$ Každý uzel kromě kořenu má jedinečného rodiče.
- Uzly se stejným rodičem jsou **sourozenci**.
- Uzel, který nemá potomky, se nazývá **list**. Ostatní uzly jsou **vnitřní**.
- **Stupeň** vnitřního uzlu je počet jeho synů (rodič se nepočítá)!!!!!!
- k -**ární strom**: každý vnitřní uzel má stupeň **nejvýše** k .

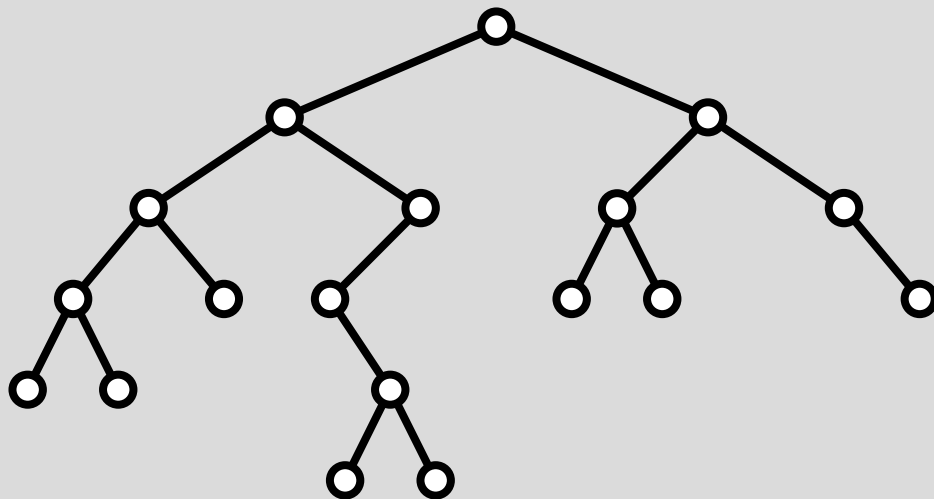
Reprezentace stromu spojovými strukturami

```
/* definice uživatelského typu uzel ve spojovém seznamu */  
typedef struct Vrchol {  
    int hodnota;  
    struct Vrchol* levy;  
    struct Vrchol* pravy;  
} Vrchol;  
  
/* struktura pro reprezentaci grafu */  
struct strom {  
    struct Vrchol* koren; // ukazatel na kořen stromu  
};
```

Hloubka kořenového stromu

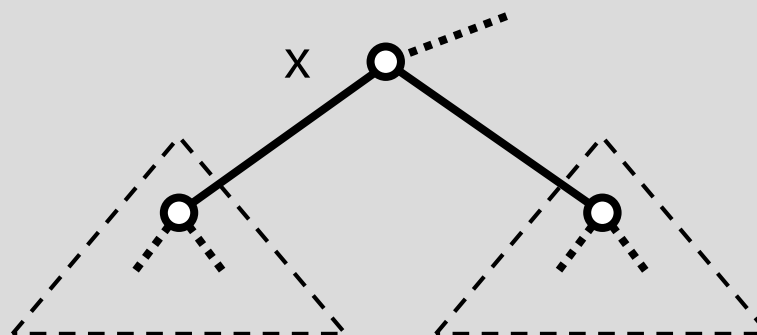


Binární kořenové stromy



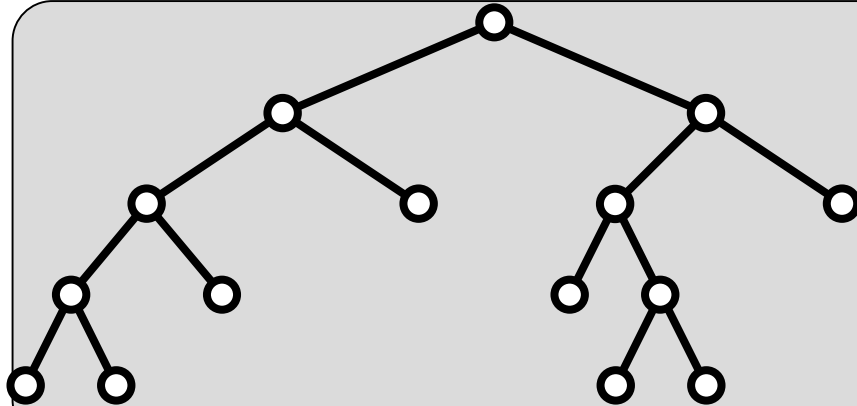
binární (kořenový!!) strom
0, 1 nebo 2 následníci

binary (rooted!!) tree
0, 1 or 2 successors



podstrom uzlu x		levý		pravý
subtree of node x		left		right

Pravidelné a vyvážené stromy

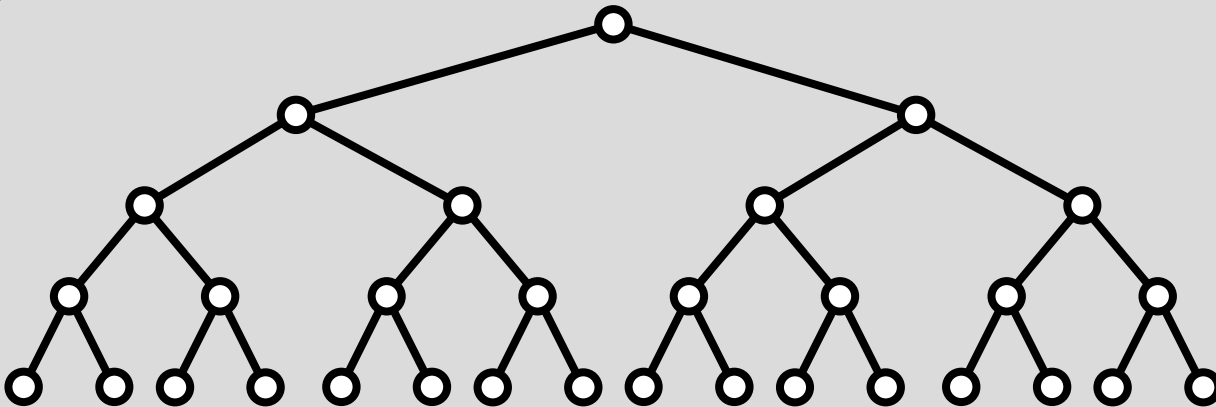


pravidelný binární strom

0 nebo 2 následníci

regular binary tree

0 or 2 successors



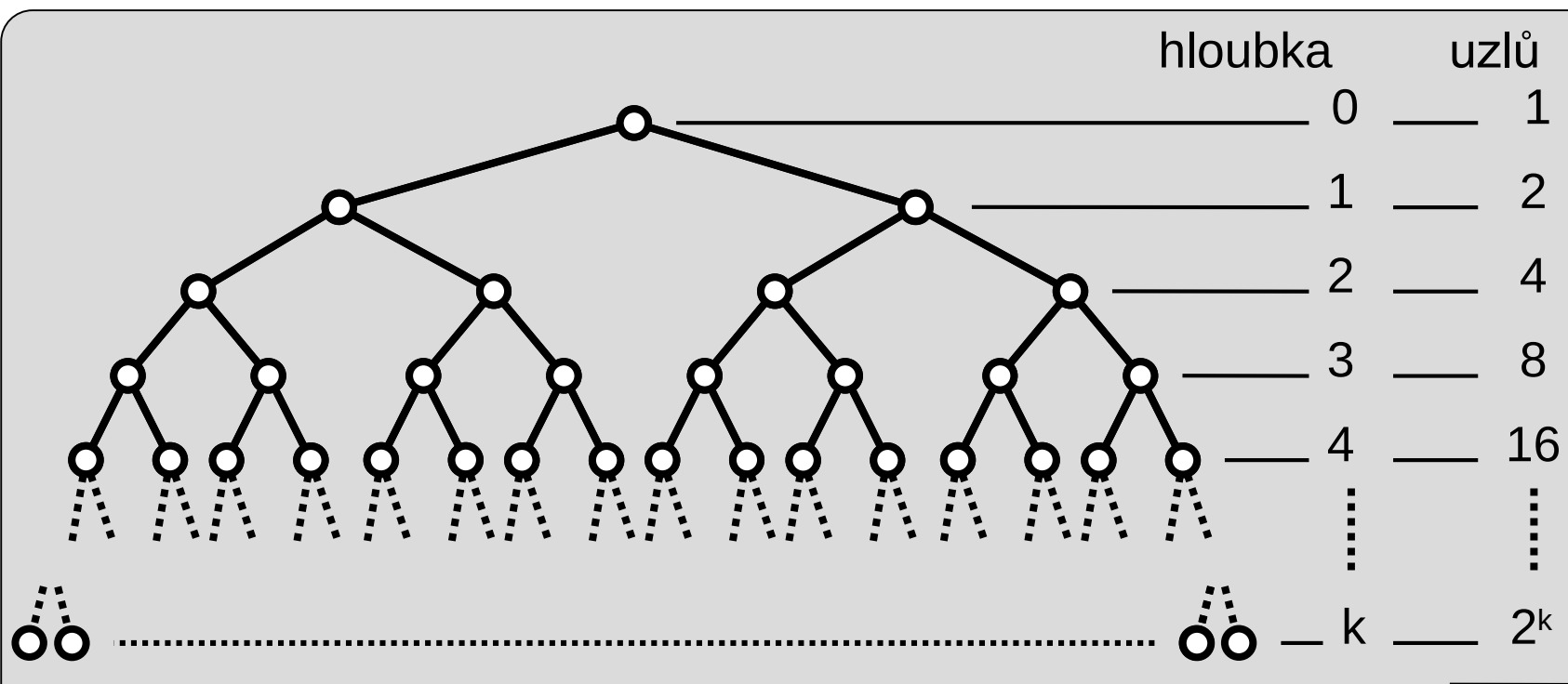
vyvážený strom

balanced tree

Hloubky všech listů jsou (víceméně) stejné.

All leaf depths are (more or less) equal.

Hloubka vyváženého stromu

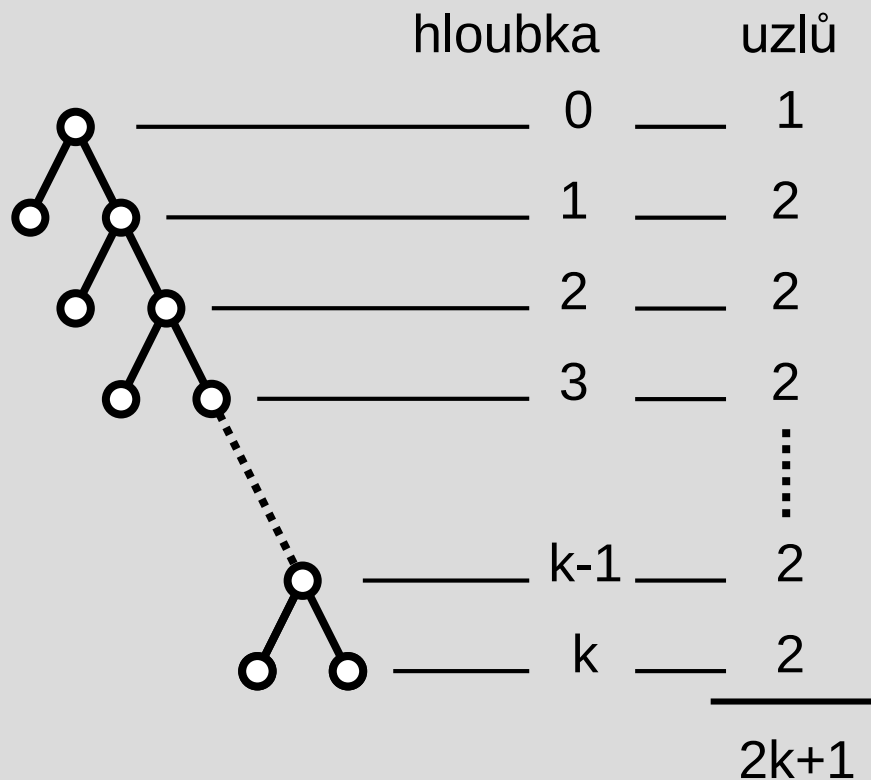


$$(2^{(\text{hloubka stromu})+1} - 1) \sim \text{uzlů}$$

$$\text{hloubka stromu} \sim \log_2(\text{uzlů} + 1) - 1$$

vyvážený strom: $\text{hloubka} \sim \log_2(\text{uzlů})$

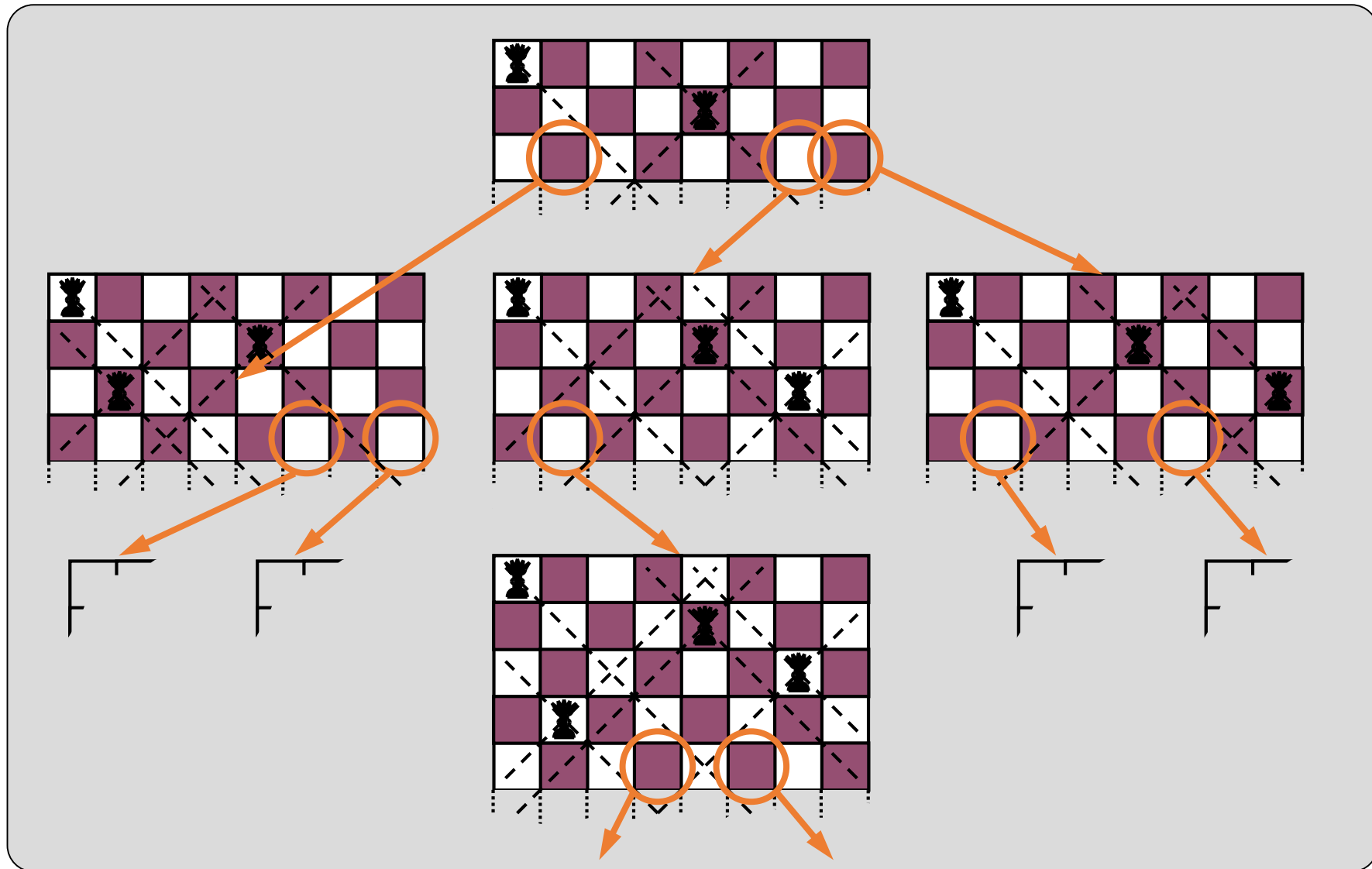
Hloubka nevyváženého stromu



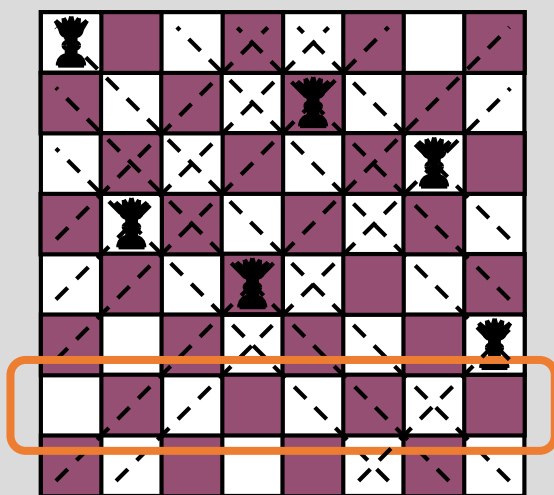
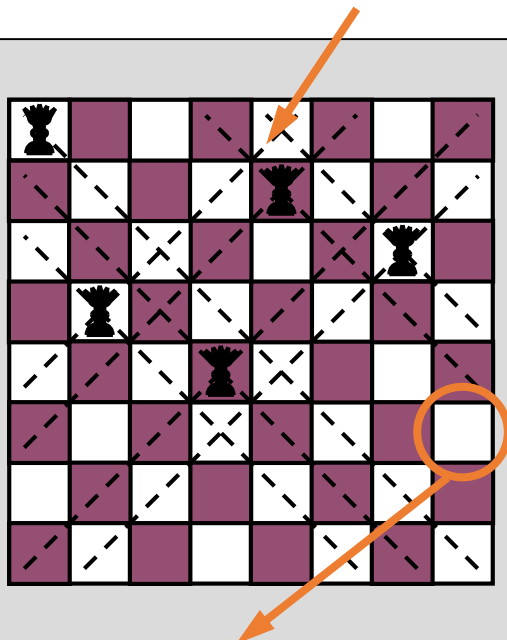
$$2(\text{hloubka})+1 \sim \text{uzlů}$$
$$\text{hloubka} \sim (\text{uzlů}-1)/2$$

extrémně nevyvážený pravidelný strom: $\text{hloubka} \sim (\text{uzlů}-1)/2$

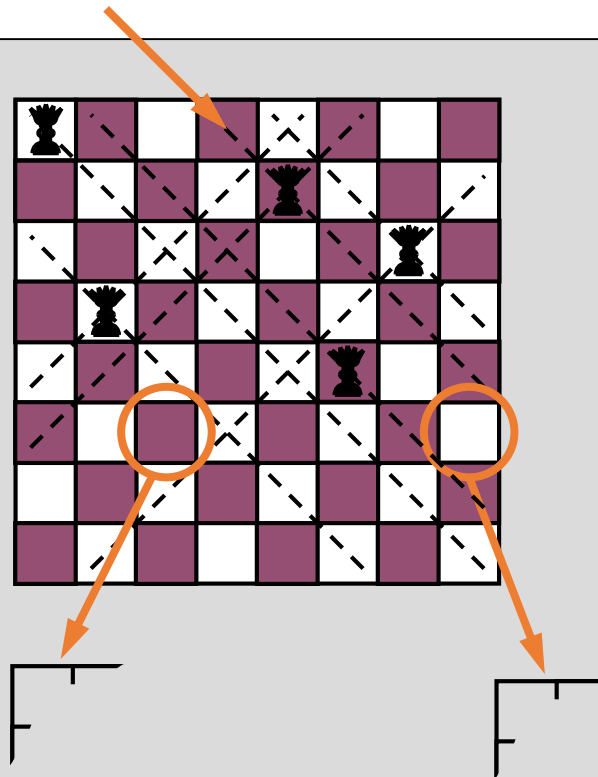
Snadné prohledávání s návratem (backtrack)



Snadné prohledávání s návratem (backtrack)



Stop!



Snadné prohledávání s návratem (backtrack)

N	Počet řešení	Počet testovaných pozic dámy		Zrychlení
		Hrubá síla (N^N)	Backtrack	
4	2	256	240	1.07
5	10	3 125	1 100	2.84
6	4	46 656	5 364	8.70
7	40	823 543	25 088	32.83
8	92	16 777 216	125 760	133.41
9	352	387 420 489	651 402	594.75
10	724	10 000 000 000	3 481 500	2 872.33
11	2 680	285 311 670 611	19 873 766	14 356.20
12	14 200	8 916 100 448 256	121 246 416	73 537.00

Tab.: Rychlosti řešení problému osmi dam

Snadné prohledávání s návratem (backtrack)

```
int posOK( int x, int y) {  
    int i;  
    for (i = 0; i < x; i++)  
        if ((xPosArr[i] == y) || // stejná řada  
            (abs(x-i) == abs(xPosArr[i]-y) )) // nebo diagonála  
            return 0;  
    return 1;  
}  
void tryPutColumn(int y) {  
    int x;  
    if (y >= N ) print_yPosArr(); // řešení  
    else  
        for (x = 0; x < N; x++) // testuj sloupce  
            if (posOK(y, x) == 1) { // když je volno,  
                xPosArr[y] = x; // dej tam dámu  
                tryPutColumn(y + 1); // a volej rekurzi  
            }  
}
```

Call: tryPutColumn(0);

Závěr

- Dynamické struktury jsou mocným nástrojem v jazyce C pro efektivní správu paměti a práci s daty. Je důležité správně alokovat a uvolňovat paměť a používat vhodné datové struktury podle konkrétních potřeb programu.

The End